

Nama : 1. Andre Hadiman Rotua Parhusip (122450108)
2. Nabyla Sharfina (123450008)
3. Arini Puteri Elandra (123450069)
4. Muhammad Fadil Alfaizi (123450115)

Kelompok : 14

Dosen Pembimbing : Yuliana M.Sc

MISI 2
DESAIN FISIKAL DAN DEVELOPMENT
FAKULTAS TEKNOLOGI INDUSTRI

1. Step 1: Physical Database Design

Tujuan: Mengimplementasikan dimensional model ke SQL Server

Aktivitas:

1.1. Database Setup

```
-- =====  
-- 01 - CREATE DATABASE FOR DATA MART FTI  
-- =====  
  
CREATE DATABASE DM_FTI_DW  
ON PRIMARY  
(  
    NAME = N'DM_FTI_DW_Data',  
    FILENAME = N'D:\DataWarehouse\DM_FTI_DW_Data.mdf',  
    SIZE = 512MB,  
    MAXSIZE = UNLIMITED,  
    FILEGROWTH = 128MB  
)  
LOG ON  
(  
    NAME = N'DM_FTI_DW_Log',  
    FILENAME = N'E:\DataWarehouseLogs\DM_FTI_DW_Log.ldf',  
    SIZE = 256MB,  
    MAXSIZE = 2GB,  
    FILEGROWTH = 64MB  
);  
GO  
  
USE DM_FTI_DW;  
GO
```

1.2. Create Dimension Tables

```
-- Dimension Table: Dim_ProgramStudi
CREATE TABLE dbo.Dim_ProgramStudi (
    sk_prodi INT IDENTITY(1,1) PRIMARY KEY,
    id_prodi INT NOT NULL UNIQUE,
    nama_prodi VARCHAR(150) NOT NULL,
    jenjang VARCHAR(10) NOT NULL,
    fakultas VARCHAR(100) NOT NULL,

    created_date DATETIME DEFAULT GETDATE(),
    updated_date DATETIME DEFAULT GETDATE()
);
GO

-- Dimension Table: Dim_Mahasiswa
CREATE TABLE dbo.Dim_Mahasiswa (
    sk_mahasiswa INT IDENTITY(1,1) PRIMARY KEY,
    nim VARCHAR(20) NOT NULL UNIQUE,
    nama_mahasiswa VARCHAR(150) NOT NULL,
    jenis_kelamin CHAR(1) CHECK (jenis_kelamin IN ('L','P')),
    angkatan INT NOT NULL,
    status VARCHAR(50),

    sk_prodi INT NOT NULL,
    FOREIGN KEY (sk_prodi) REFERENCES
    dbo.Dim_ProgramStudi(sk_prodi),

    created_date DATETIME DEFAULT GETDATE(),
    updated_date DATETIME DEFAULT GETDATE()
);
GO

-- Dimension Table: Dim_Dosen
CREATE TABLE dbo.Dim_Dosen (
    sk_dosen INT IDENTITY(1,1) PRIMARY KEY,
    nidn VARCHAR(20) NOT NULL UNIQUE,
    nama_dosen VARCHAR(150) NOT NULL,
    jabatan_fungsional VARCHAR(150),
    status_kepegawaian VARCHAR(50),
```

```
    sk_prodi INT NOT NULL,  
    FOREIGN KEY (sk_prodi) REFERENCES  
dbo.Dim_ProgramStudi(sk_prodi),  
  
    created_date DATETIME DEFAULT GETDATE(),  
    updated_date DATETIME DEFAULT GETDATE()  
);  
GO
```

```
-- Dimension Table: Dim_Waktu  
CREATE TABLE dbo.Dim_Waktu (  
    sk_waktu INT PRIMARY KEY,  
    tanggal DATE NOT NULL,  
    tahun INT NOT NULL,  
    semester VARCHAR(10) NOT NULL,  
    bulan INT NOT NULL,  
    nama_bulan VARCHAR(10) NOT NULL  
);  
GO
```

```
-- Dimension Table: Dim_Prestasi  
CREATE TABLE dbo.Dim_Prestasi (  
    sk_prestasi INT IDENTITY(1,1) PRIMARY KEY,  
    id_prestasi VARCHAR(20) NOT NULL UNIQUE,  
    nama_prestasi VARCHAR(100),  
    jenis_prestasi VARCHAR(30),  
    tingkat VARCHAR(30),  
    penyelenggara VARCHAR(100),  
  
    created_date DATETIME DEFAULT GETDATE()  
);  
GO
```

```
-- Dimension Table: Dim_Anggaran  
CREATE TABLE dbo.Dim_Anggaran (  
    sk_anggaran INT IDENTITY(1,1) PRIMARY KEY,  
    id_anggaran VARCHAR(20) NOT NULL UNIQUE,  
    kategori VARCHAR(100),  
    keterangan VARCHAR(200),  
  
    created_date DATETIME DEFAULT GETDATE()  
);
```

```
GO
```

```
-- Dim_Akreditasi
```

```
CREATE TABLE dbo.Dim_Akreditasi (  
    sk_akreditasi INT IDENTITY(1,1) PRIMARY KEY,  
    id_akreditasi VARCHAR(20) NOT NULL UNIQUE,  
    status_akreditasi VARCHAR(20),  
    nilai_akreditasi DECIMAL(3,2),  
    lembaga VARCHAR(20),  
  
    created_date DATETIME DEFAULT GETDATE()  
);  
GO
```

1.3. Create Fact Tables

```
-- Fact Table: Fact_Prestasi
```

```
CREATE TABLE dbo.Fact_Prestasi (  
    sk_prestasi INT NOT NULL,  
    sk_mahasiswa INT NOT NULL,  
    sk_prodi INT NOT NULL,  
    sk_waktu INT NOT NULL,  
  
    created_date DATETIME DEFAULT GETDATE(),  
  
    FOREIGN KEY (sk_prestasi) REFERENCES  
    dbo.Dim_Prestasi(sk_prestasi),  
    FOREIGN KEY (sk_mahasiswa) REFERENCES  
    dbo.Dim_Mahasiswa(sk_mahasiswa),  
    FOREIGN KEY (sk_prodi) REFERENCES  
    dbo.Dim_ProgramStudi(sk_prodi),  
    FOREIGN KEY (sk_waktu) REFERENCES dbo.Dim_Waktu(sk_waktu)  
);  
GO
```

```
-- Fact Table: Fact_Anggaran
```

```
CREATE TABLE dbo.Fact_Anggaran (  
    sk_anggaran INT NOT NULL,  
    sk_prodi INT NOT NULL,  
    sk_waktu INT NOT NULL, -- mapped by tahun only
```

```

total_anggaran DECIMAL(18,2) NOT NULL,

created_date DATETIME DEFAULT GETDATE(),

FOREIGN KEY (sk_anggaran) REFERENCES
dbo.Dim_Anggaran(sk_anggaran),
FOREIGN KEY (sk_prodi) REFERENCES
dbo.Dim_ProgramStudi(sk_prodi),
FOREIGN KEY (sk_waktu) REFERENCES dbo.Dim_Waktu(sk_waktu)
);
GO

```

```

-- Fact Table: Fact_Akreditasi
CREATE TABLE dbo.Fact_Akreditasi (
    sk_akreditasi INT NOT NULL,
    sk_prodi INT NOT NULL,
    sk_waktu INT NOT NULL,

    created_date DATETIME DEFAULT GETDATE(),

    FOREIGN KEY (sk_akreditasi) REFERENCES
    dbo.Dim_Akreditasi(sk_akreditasi),
    FOREIGN KEY (sk_prodi) REFERENCES
    dbo.Dim_ProgramStudi(sk_prodi),
    FOREIGN KEY (sk_waktu) REFERENCES dbo.Dim_Waktu(sk_waktu)
);
GO

```

```

-- Fact Table: Fact_Akademik
CREATE TABLE dbo.Fact_Akademik (
    sk_prodi INT NOT NULL,
    sk_waktu INT NOT NULL,

    jumlah_mahasiswa_baru INT,
    rata_rata_ipk DECIMAL(4,2),

    created_date DATETIME DEFAULT GETDATE(),

    FOREIGN KEY (sk_prodi) REFERENCES
    dbo.Dim_ProgramStudi(sk_prodi),
    FOREIGN KEY (sk_waktu) REFERENCES dbo.Dim_Waktu(sk_waktu)
);

```

```

GO

-- Fact Table: Fact_Dosen
CREATE TABLE dbo.Fact_Dosen (
    sk_prodi INT NOT NULL,
    sk_waktu INT NOT NULL,

    jumlah_dosen INT,
    jumlah_mahasiswa INT,
    rasio_dosen_mahasiswa DECIMAL(6,3),

    created_date DATETIME DEFAULT GETDATE(),

    FOREIGN KEY (sk_prodi) REFERENCES
    dbo.Dim_ProgramStudi(sk_prodi),
    FOREIGN KEY (sk_waktu) REFERENCES dbo.Dim_Waktu(sk_waktu)
);
GO

```

2. Step 2: Indexing Strategy

Tujuan: Mengoptimalkan query performance dengan indexing yang tepat

Aktivitas:

2.1. Clustered Index on Fact Table

```

-- Clustered Index for Fact_Prestasi (grain: mahasiswa-prestasi-waktu-prodi)
CREATE CLUSTERED INDEX CIX_Fact_Prestasi
ON dbo.Fact_Prestasi (sk_mahasiswa, sk_prestasi, sk_waktu, sk_prodi);
GO

CREATE CLUSTERED INDEX CIX_Fact_Anggaran
ON dbo.Fact_Anggaran (sk_prodi, sk_waktu, sk_anggaran);
GO

CREATE CLUSTERED INDEX CIX_Fact_Akreditasi
ON dbo.Fact_Akreditasi (sk_prodi, sk_waktu, sk_akreditasi);
GO

CREATE CLUSTERED INDEX CIX_Fact_Akademik
ON dbo.Fact_Akademik (sk_prodi, sk_waktu);
GO

CREATE CLUSTERED INDEX CIX_Fact_Dosen
ON dbo.Fact_Dosen (sk_prodi, sk_waktu);
GO

```

2.2. Non-Clustered Indexes

```
-----  
-- Index untuk Foreign Keys pada Fact_Prestasi (join optimization)  
-----
```

```
CREATE NONCLUSTERED INDEX IX_Fact_Prestasi_sk_waktu  
ON dbo.Fact_Prestasi (sk_waktu);
```

```
CREATE NONCLUSTERED INDEX IX_Fact_Prestasi_sk_prodi  
ON dbo.Fact_Prestasi (sk_prodi);
```

```
CREATE NONCLUSTERED INDEX IX_Fact_Prestasi_sk_mahasiswa  
ON dbo.Fact_Prestasi (sk_mahasiswa);  
GO
```

```
-----  
-- Index untuk Foreign Keys pada Fact_Anggaran  
-----
```

```
CREATE NONCLUSTERED INDEX IX_Fact_Anggaran_sk_waktu  
ON dbo.Fact_Anggaran (sk_waktu);
```

```
CREATE NONCLUSTERED INDEX IX_Fact_Anggaran_sk_prodi  
ON dbo.Fact_Anggaran (sk_prodi);  
GO
```

```
-----  
-- Index untuk Foreign Keys pada Fact_Akreditasi  
-----
```

```
CREATE NONCLUSTERED INDEX IX_Fact_Akreditasi_sk_waktu  
ON dbo.Fact_Akreditasi (sk_waktu);
```

```
CREATE NONCLUSTERED INDEX IX_Fact_Akreditasi_sk_prodi  
ON dbo.Fact_Akreditasi (sk_prodi);  
GO
```

```
-----  
-- Index untuk Foreign Keys pada Fact_Akademik  
-----
```

```
CREATE NONCLUSTERED INDEX IX_Fact_Akademik_sk_waktu  
ON dbo.Fact_Akademik (sk_waktu);
```

```
CREATE NONCLUSTERED INDEX IX_Fact_Akademik_sk_prodi
```

```

ON dbo.Fact_Akademik (sk_prodi);
GO

-----
-- Index untuk Foreign Keys pada Fact_Dosen
-----
CREATE NONCLUSTERED INDEX IX_Fact_Dosen_sk_waktu
ON dbo.Fact_Dosen (sk_waktu);

CREATE NONCLUSTERED INDEX IX_Fact_Dosen_sk_prodi
ON dbo.Fact_Dosen (sk_prodi);
GO

-----
-- Covering index untuk common queries (Prestasi per Prodi per Waktu)
-----
CREATE NONCLUSTERED INDEX IX_Covering_Prestasi_Prodi_Waktu
ON dbo.Fact_Prestasi (sk_waktu, sk_prodi);
GO

-----
-- Covering index untuk laporan Anggaran per Prodi per Tahun
-----
CREATE NONCLUSTERED INDEX IX_Covering_Anggaran_Prodi_Waktu
ON dbo.Fact_Anggaran (sk_waktu, sk_prodi)
INCLUDE (total_anggaran);
GO

-----
-- Covering index untuk laporan Akreditasi per Prodi per Tahun
-----
CREATE NONCLUSTERED INDEX IX_Covering_Akreditasi_Prodi_Waktu
ON dbo.Fact_Akreditasi (sk_waktu, sk_prodi);
GO

-----
-- Covering index untuk laporan Rasio Dosen per Prodi per Tahun
-----
CREATE NONCLUSTERED INDEX IX_Covering_Dosen_Prodi_Waktu
ON dbo.Fact_Dosen (sk_waktu, sk_prodi)
INCLUDE (jumlah_dosen, jumlah_mahasiswa, rasio_dosen_mahasiswa);
GO

```

2.3. Columnstore Index (For Large Fact Tables)


```

-- Columnstore index untuk analisis prestasi mahasiswa
CREATE NONCLUSTERED COLUMNSTORE INDEX CS_Fact_Prestasi
ON dbo.Fact_Prestasi
(
    sk_waktu,
    sk_prodi,
    sk_mahasiswa
);
GO

-- Columnstore index untuk analisis anggaran per tahun/prodi
CREATE NONCLUSTERED COLUMNSTORE INDEX CS_Fact_Anggaran
ON dbo.Fact_Anggaran
(
    sk_waktu,
    sk_prodi,
    total_anggaran
);
GO

-- Columnstore index untuk analisis kualitas akreditasi setiap prodi
CREATE NONCLUSTERED COLUMNSTORE INDEX CS_Fact_Akreditasi
ON dbo.Fact_Akreditasi
(
    sk_waktu,
    sk_prodi
);
GO

-- Columnstore index untuk analisis akademik mahasiswa per waktu
CREATE NONCLUSTERED COLUMNSTORE INDEX CS_Fact_Akademik
ON dbo.Fact_Akademik
(
    sk_waktu,
    sk_prodi,
    jumlah_mahasiswa_baru,
    rata_rata_ipk
);
GO

-- Columnstore index untuk analisis rasio dosen_mahasiswa per waktu
CREATE NONCLUSTERED COLUMNSTORE INDEX CS_Fact_Dosen
ON dbo.Fact_Dosen
(
    sk_waktu,
    sk_prodi,

```

```
    jumlah_dosen,  
    jumlah_mahasiswa,  
    rasio_dosen_mahasiswa  
);  
GO
```

3. Step 3: Partitioning Strategy

Tujuan: Meningkatkan manageability dan performance untuk large tables

Aktivitas:

```
-- =====  
-- Partitioning by Academic Year ranges  
-- =====  
  
CREATE PARTITION FUNCTION PF_AcademicYear (INT)  
AS RANGE RIGHT FOR VALUES  
(  
    20200101,  
    20210101,  
    20220101,  
    20230101,  
    20240101,  
    20250101  
);  
GO  
  
CREATE PARTITION SCHEME PS_AcademicYear  
AS PARTITION PF_AcademicYear  
ALL TO ([PRIMARY]);  
GO  
  
CREATE TABLE dbo.Fact_Prestasi_Partitioned  
(  
    sk_prestasi INT NOT NULL,  
    sk_mahasiswa INT NOT NULL,  
    sk_prodi INT NOT NULL,  
    sk_waktu INT NOT NULL,  
  
    created_date DATETIME DEFAULT GETDATE(),  
  
    FOREIGN KEY (sk_prestasi) REFERENCES dbo.Dim_Prestasi(sk_prestasi),  
    FOREIGN KEY (sk_mahasiswa) REFERENCES  
dbo.Dim_Mahasiswa(sk_mahasiswa),  
    FOREIGN KEY (sk_prodi) REFERENCES dbo.Dim_ProgramStudi(sk_prodi),  
    FOREIGN KEY (sk_waktu) REFERENCES dbo.Dim_Waktu(sk_waktu)
```

```

)
ON PS_AcademicYear(sk_waktu);
GO

CREATE TABLE dbo.Fact_Anggaran_Partitioned
(
    sk_anggaran INT NOT NULL,
    sk_prodi INT NOT NULL,
    sk_waktu INT NOT NULL,

    total_anggaran DECIMAL(18,2),
    created_date DATETIME DEFAULT GETDATE(),

    FOREIGN KEY (sk_anggaran) REFERENCES dbo.Dim_Anggaran(sk_anggaran),
    FOREIGN KEY (sk_prodi) REFERENCES dbo.Dim_ProgramStudi(sk_prodi),
    FOREIGN KEY (sk_waktu) REFERENCES dbo.Dim_Waktu(sk_waktu)
)
ON PS_AcademicYear(sk_waktu);
GO

CREATE TABLE dbo.Fact_Akreditasi_Partitioned
(
    sk_akreditasi INT NOT NULL,
    sk_prodi INT NOT NULL,
    sk_waktu INT NOT NULL,

    created_date DATETIME DEFAULT GETDATE(),

    FOREIGN KEY (sk_akreditasi) REFERENCES dbo.Dim_Akreditasi(sk_akreditasi),
    FOREIGN KEY (sk_prodi) REFERENCES dbo.Dim_ProgramStudi(sk_prodi),
    FOREIGN KEY (sk_waktu) REFERENCES dbo.Dim_Waktu(sk_waktu)
)
ON PS_AcademicYear(sk_waktu);
GO

CREATE TABLE dbo.Fact_Akademik_Partitioned
(
    sk_prodi INT NOT NULL,
    sk_waktu INT NOT NULL,

    jumlah_mahasiswa_baru INT,
    rata_rata_ipk DECIMAL(4,2),
    created_date DATETIME DEFAULT GETDATE(),

    FOREIGN KEY (sk_prodi) REFERENCES dbo.Dim_ProgramStudi(sk_prodi),
    FOREIGN KEY (sk_waktu) REFERENCES dbo.Dim_Waktu(sk_waktu)
)

```

```

)
ON PS_AcademicYear(sk_waktu);
GO

CREATE TABLE dbo.Fact_Dosen_Partitioned
(
    sk_prodi INT NOT NULL,
    sk_waktu INT NOT NULL,

    jumlah_dosen INT,
    jumlah_mahasiswa INT,
    rasio_dosen_mahasiswa DECIMAL(6,3),
    created_date DATETIME DEFAULT GETDATE(),

    FOREIGN KEY (sk_prodi) REFERENCES dbo.Dim_ProgramStudi(sk_prodi),
    FOREIGN KEY (sk_waktu) REFERENCES dbo.Dim_Waktu(sk_waktu)
)
ON PS_AcademicYear(sk_waktu);
GO

```

4. Step 4: ETL Design

Tujuan: Merancang proses Extract, Transform, Load yang efisien

Aktivitas:

4.1. ETL Architecture Design

Sumber Data (Website / CSV / Excel / Input Manual / Sistem Kepegawaian) → Extract Staging Schema (stg.*) — data mentah, transformasi minimal → Cleansing & Transformasi (Integration Layer) Integration Schema (int.*) — data dinormalisasi, business keys, penanganan SCD → Load Data Warehouse (dbo.*) — dimensi (SCD Tipe 1/2), tabel fakta (partitioned & indexed) → Konsumsi (laporan / dashboard / OLAP)

Tujuan utama:

- Pisahkan ekstrak mentah di **stg** untuk audit dan kemudahan reprocessing.
- Pastikan proses idempotent (UPSERT, lookup, hash untuk deteksi perubahan).
- Terapkan SCD Type 2 untuk Dim_Mahasiswa (dan dimensi lain jika perlu histori).
- Gunakan batch load + partition switching pada tabel fakta besar untuk performa.

4.2. Alur ETL & Tanggung Jawab

4.2.1. Source → Staging (Extract)

- Frekuensi: sesuai sumber (harian / bulanan / tahunan).

- Aktivitas: ingest file (CSV/XLSX), scrapping halaman web yang relevan, validasi manual untuk input manual.
- Tambahkan metadata audit: source_name, source_file, extract_ts, row_hash (HASHBYTES) pada tiap tabel staging.

4.2.2. **Staging → Integration (Transform & Cleanse)**

- Validasi format NIM/NIDN (trim, uppercase).
- Deduplication berdasarkan row_hash + business key.
- Normalisasi nilai kategorikal (mis. jenis_kelamin → 'L'/'P').
- Mapping kode program sumber ke id_prodi canonical.
- Resolusi surrogate key melalui lookup ke Dim_*.

4.2.3. **Integration → Data Warehouse (Load)**

- Dimensi: SCD Type 2 untuk Dim_Mahasiswa; Type 1 untuk Dim_ProgramStudi (default).
- Gunakan lookup untuk resolusi SK pada load fakta.
- Fakta: insert ke tabel fakta partitioned; gunakan SWITCH untuk bulk loads bila tersedia.
- Nonclustered index dapat dinonaktifkan saat load besar lalu dibangun ulang setelahnya.

4.3. Create Staging Tables

```
CREATE SCHEMA stg;
GO

CREATE TABLE stg.Mahasiswa (
    source_file VARCHAR(255),
    nim VARCHAR(20),
    nama_mahasiswa VARCHAR(200),
    jenis_kelamin CHAR(1),
    angkatan INT,
    status VARCHAR(20),
    kode_prodi VARCHAR(20),
    extract_ts DATETIME DEFAULT GETDATE(),
    row_hash BINARY(16)
);
GO

CREATE TABLE stg.Prestasi (
    source_file VARCHAR(255),
    id_prestasi VARCHAR(50),
    nim VARCHAR(20),
    kode_prodi VARCHAR(20),
    tanggal_prestasi DATE,
```

```
extract_ts DATETIME DEFAULT GETDATE(),  
row_hash BINARY(16)  
);  
GO
```

```
CREATE TABLE stg.Anggaran (  
    source_file VARCHAR(255),  
    id_anggaran VARCHAR(50),  
    kode_prodi VARCHAR(20),  
    total_anggaran DECIMAL(18,2),  
    tahun INT,  
    extract_ts DATETIME DEFAULT GETDATE(),  
    row_hash BINARY(16)  
);  
GO
```

```
CREATE TABLE stg.Akreditasi (  
    source_file VARCHAR(255),  
    id_akreditasi VARCHAR(50),  
    id_prodi VARCHAR(20),  
    status_akreditasi VARCHAR(50),  
    nilai_akreditasi DECIMAL(3,2),  
    lembaga VARCHAR(100),  
    tahun INT,  
    extract_ts DATETIME DEFAULT GETDATE(),  
    row_hash BINARY(16)  
);  
GO
```

```
CREATE TABLE stg.ProgramStudi (  
    source_file VARCHAR(255),  
    id_prodi VARCHAR(20),  
    nama_prodi VARCHAR(200),  
    jenjang VARCHAR(20),  
    fakultas VARCHAR(100),  
    extract_ts DATETIME DEFAULT GETDATE(),  
    row_hash BINARY(16)  
);  
GO
```

```
CREATE TABLE stg.Dosen (  
    source_file VARCHAR(255),  
    nidn VARCHAR(20),  
    nama_dosen VARCHAR(200),  
    kode_prodi VARCHAR(20),  
    tahun INT,
```

```

jumlah_mahasiswa INT,
jumlah_dosen INT,
rasio_dosen_mahasiswa DECIMAL(6,3),
extract_ts DATETIME DEFAULT GETDATE(),
row_hash BINARY(16)
);
GO

CREATE TABLE stg.Akademik (
    source_file VARCHAR(255),
    kode_prodi VARCHAR(20),
    tahun INT,
    jumlah_mahasiswa_baru INT,
    rata_rata_ipk DECIMAL(4,2),
    extract_ts DATETIME DEFAULT GETDATE(),
    row_hash BINARY(16)
);
GO

```

4.4. ETL Mapping Document

Source	Source Column	Target	Target Column	Transformation

5. Step 5: ETL Implementation

Tujuan: Mengimplementasikan ETL menggunakan SSIS atau T-SQL scripts

Opsi 1: Menggunakan SSIS (Recommended)

5.1. Create SSIS Project

- Buka SQL Server Data Tools (SSDT)
- Create new Integration Services Project
- Beri nama: ETL_[UnitName]_DW

5.2. Package 1: Load Dimensions

- a. Data Flow Task: Extract dari source
 - b. Derived Column: Transformasi data
 - c. Lookup Transformation: SCD Type 2 handling
 - d. Slowly Changing Dimension: Insert/Update dimension
- 5.3. Package 2: Load Facts
- a. Data Flow Task: Extract enrollment data
 - b. Lookup Transformations: Resolve dimension keys
 - c. Derived Column: Calculate measures
 - d. OLE DB Destination: Load to fact table
- 5.4. Master Package
- a. Sequence Container: Truncate staging
 - b. Execute SQL Task: Disable indexes
 - c. Execute Package Task: Load dimensions
 - d. Execute Package Task: Load facts
 - e. Execute SQL Task: Rebuild indexes
 - f. Execute SQL Task: Update statistics

Opsi 2: Menggunakan T-SQL Stored Procedures

```
USE DM_FTI_DW;
GO

-- Dim ProgramStudi
MERGE dbo.Dim_ProgramStudi AS tgt
USING (
    SELECT DISTINCT id_prodi, nama_prodi, jenjang, fakultas FROM
    stg.ProgramStudi
) AS src
ON tgt.id_prodi = src.id_prodi
WHEN MATCHED THEN UPDATE SET
    tgt.nama_prodi = src.nama_prodi,
    tgt.jenjang = src.jenjang,
    tgt.fakultas = src.fakultas,
    tgt.updated_date = GETDATE()
WHEN NOT MATCHED THEN
    INSERT (id_prodi, nama_prodi, jenjang, fakultas)
    VALUES (src.id_prodi, src.nama_prodi, src.jenjang, src.fakultas);
GO

-- Dim Mahasiswa
MERGE dbo.Dim_Mahasiswa AS tgt
USING (
    SELECT nim, nama_mahasiswa, jenis_kelamin, angkatan, status, kode_prodi
    FROM stg.Mahasiswa
```



```

) AS src
ON tgt.nim = src.nim
WHEN MATCHED THEN UPDATE SET
    tgt.nama_mahasiswa = src.nama_mahasiswa,
    tgt.jenis_kelamin = src.jenis_kelamin,
    tgt.angkatan = src.angkatan,
    tgt.status = src.status,
    tgt.sk_prodi = (SELECT sk_prodi FROM dbo.Dim_ProgramStudi WHERE id_prodi
= src.kode_prodi),
    tgt.updated_date = GETDATE()
WHEN NOT MATCHED THEN
    INSERT (nim, nama_mahasiswa, jenis_kelamin, angkatan, status, sk_prodi)
    VALUES (
        src.nim, src.nama_mahasiswa, src.jenis_kelamin, src.angkatan, src.status,
        (SELECT sk_prodi FROM dbo.Dim_ProgramStudi WHERE id_prodi =
src.kode_prodi)
    );
GO

```

```

-- Dim Prestasi
MERGE dbo.Dim_Prestasi AS tgt
USING (SELECT DISTINCT id_prestasi FROM stg.Prestasi) AS src
ON tgt.id_prestasi = src.id_prestasi
WHEN NOT MATCHED THEN
    INSERT (id_prestasi) VALUES (src.id_prestasi);
GO

```

```

-- Dim Anggaran
MERGE dbo.Dim_Anggaran AS tgt
USING (SELECT DISTINCT id_anggaran FROM stg.Anggaran) AS src
ON tgt.id_anggaran = src.id_anggaran
WHEN NOT MATCHED THEN
    INSERT (id_anggaran) VALUES (src.id_anggaran);
GO

```

```

-- Dim Akreditasi
MERGE dbo.Dim_Akreditasi AS tgt
USING (
    SELECT DISTINCT id_akreditasi, status_akreditasi, nilai_akreditasi, lembaga
FROM stg.Akreditasi
) AS src
ON tgt.id_akreditasi = src.id_akreditasi
WHEN MATCHED THEN UPDATE SET
    tgt.status_akreditasi = src.status_akreditasi,
    tgt.nilai_akreditasi = src.nilai_akreditasi,
    tgt.lembaga = src.lembaga

```

```
WHEN NOT MATCHED THEN
    INSERT (id_akreditasi, status_akreditasi, nilai_akreditasi, lembaga)
    VALUES (src.id_akreditasi, src.status_akreditasi, src.nilai_akreditasi, src.lembaga);
GO
```

```
-- Dim Dosen
MERGE dbo.Dim_Dosen AS tgt
USING (
    SELECT DISTINCT nidn, nama_dosen, kode_prodi FROM stg.Dosen
) AS src
ON tgt.nidn = src.nidn
WHEN MATCHED THEN UPDATE SET
    tgt.nama_dosen = src.nama_dosen,
    tgt.sk_prodi = (SELECT sk_prodi FROM dbo.Dim_ProgramStudi WHERE id_prodi
= src.kode_prodi),
    tgt.updated_date = GETDATE()
WHEN NOT MATCHED THEN
    INSERT (nidn, nama_dosen, sk_prodi)
    VALUES (
        src.nidn, src.nama_dosen,
        (SELECT sk_prodi FROM dbo.Dim_ProgramStudi WHERE id_prodi =
src.kode_prodi)
    );
GO
```

```
-- Dim Waktu (generate per year from staging)
INSERT INTO dbo.Dim_Waktu (sk_waktu, tanggal, tahun, semester, bulan,
nama_bulan)
SELECT DISTINCT
    CAST(CONCAT(tahun, '0101') AS INT) AS sk_waktu,
    CAST(CONCAT(tahun, '-01-01') AS DATE) AS tanggal,
    tahun,
    'Genap' AS semester,
    1 AS bulan,
    'Januari' AS nama_bulan
FROM (
    SELECT tahun FROM stg.Anggaran
    UNION
    SELECT tahun FROM stg.Akreditasi
    UNION
    SELECT tahun FROM stg.Dosen
    UNION
    SELECT tahun FROM stg.Akademik
) AS x
WHERE NOT EXISTS (SELECT 1 FROM dbo.Dim_Waktu w WHERE w.tahun =
x.tahun);
```

GO

-- Fact_Prestasi

INSERT INTO dbo.Fact_Prestasi (sk_prestasi, sk_mahasiswa, sk_prodi, sk_waktu)

SELECT

dp.sk_prestasi,
dm.sk_mahasiswa,
pr.sk_prodi,
dw.sk_waktu

FROM stg.Prestasi s

JOIN dbo.Dim_Prestasi dp ON dp.id_prestasi = s.id_prestasi

JOIN dbo.Dim_Mahasiswa dm ON dm.nim = s.nim

JOIN dbo.Dim_ProgramStudi pr ON pr.id_prodi = s.kode_prodi

JOIN dbo.Dim_Waktu dw ON dw.tanggal = s.tanggal_prestasi;

GO

-- Fact_Anggaran

INSERT INTO dbo.Fact_Anggaran (sk_anggaran, sk_prodi, sk_waktu, total_anggaran)

SELECT

da.sk_anggaran,
dp.sk_prodi,
dw.sk_waktu,
s.total_anggaran

FROM stg.Anggaran s

JOIN dbo.Dim_Anggaran da ON da.id_anggaran = s.id_anggaran

JOIN dbo.Dim_ProgramStudi dp ON dp.id_prodi = s.kode_prodi

JOIN dbo.Dim_Waktu dw ON dw.tahun = s.tahun;

GO

-- Fact_Akreditasi

INSERT INTO dbo.Fact_Akreditasi (sk_akreditasi, sk_prodi, sk_waktu,
nilai_akreditasi)

SELECT

da.sk_akreditasi,
dp.sk_prodi,
dw.sk_waktu,
s.nilai_akreditasi

FROM stg.Akreditasi s

JOIN dbo.Dim_Akreditasi da ON da.id_akreditasi = s.id_akreditasi

JOIN dbo.Dim_ProgramStudi dp ON dp.id_prodi = s.kode_prodi

JOIN dbo.Dim_Waktu dw ON dw.tahun = s.tahun;

GO

-- Fact_Akademik

INSERT INTO dbo.Fact_Akademik (sk_prodi, sk_waktu, jumlah_mahasiswa_baru,
rata_rata_ipk)

```

SELECT
    dp.sk_prodi,
    dw.sk_waktu,
    s.jumlah_mahasiswa_baru,
    s.rata_rata_ipk
FROM stg.Akademik s
JOIN dbo.Dim_ProgramStudi dp ON dp.id_prodi = s.kode_prodi
JOIN dbo.Dim_Waktu dw ON dw.tahun = s.tahun;
GO

-- Fact_Dosen
INSERT INTO dbo.Fact_Dosen (sk_prodi, sk_waktu, jumlah_dosen,
jumlah_mahasiswa, rasio_dosen_mahasiswa)
SELECT
    dp.sk_prodi,
    dw.sk_waktu,
    s.jumlah_dosen,
    s.jumlah_mahasiswa,
    s.rasio_dosen_mahasiswa
FROM stg.Dosen s
JOIN dbo.Dim_ProgramStudi dp ON dp.id_prodi = s.kode_prodi
JOIN dbo.Dim_Waktu dw ON dw.tahun = s.tahun;
GO

```

6. Step 6: Data Quality Assurance

Tujuan: Memastikan kualitas data yang dimuat ke data warehouse

Aktivitas:

6.1. Data Quality Checks

```

-- Completeness
-- Missing key attributes
SELECT COUNT(*) AS MissingNIM
FROM dbo.Dim_Mahasiswa
WHERE nim IS NULL OR nim = "";
GO

SELECT COUNT(*) AS MissingNamaMahasiswa
FROM dbo.Dim_Mahasiswa
WHERE nama_mahasiswa IS NULL OR nama_mahasiswa = "";
GO

-- Missing Prestasi Dates in staging
SELECT COUNT(*) AS MissingTanggalPrestasi
FROM stg.Prestasi
WHERE tanggal_prestasi IS NULL;

```

GO

-- Missing Anggaran Amount

```
SELECT COUNT(*) AS MissingTotalAnggaran
FROM stg.Anggaran
WHERE total_anggaran IS NULL;
GO
```

-- Duplicate Checks

-- Duplicate Mahasiswa (should not happen)

```
SELECT nim, COUNT(*) AS dupe_count
FROM dbo.Dim_Mahasiswa
GROUP BY nim
HAVING COUNT(*) > 1;
GO
```

-- Duplicate Prestasi IDs

```
SELECT id_prestasi, COUNT(*) AS dupe_count
FROM dbo.Dim_Prestasi
GROUP BY id_prestasi
HAVING COUNT(*) > 1;
GO
```

-- Duplicate Prodi

```
SELECT id_prodi, COUNT(*) AS dupe_count
FROM dbo.Dim_ProgramStudi
GROUP BY id_prodi
HAVING COUNT(*) > 1;
GO
```

-- Referential Integrity Checks

-- Fact_Prestasi → Dim_Mahasiswa

```
SELECT COUNT(*) AS Orphan_FactPrestasi_Mahasiswa
FROM dbo.Fact_Prestasi fp
LEFT JOIN dbo.Dim_Mahasiswa dm ON fp.sk_mahasiswa =
dm.sk_mahasiswa
WHERE dm.sk_mahasiswa IS NULL;
GO
```

-- Fact_Prestasi → Dim_Prestasi

```
SELECT COUNT(*) AS Orphan_FactPrestasi_Prestasi
FROM dbo.Fact_Prestasi fp
LEFT JOIN dbo.Dim_Prestasi dp ON fp.sk_prestasi = dp.sk_prestasi
WHERE dp.sk_prestasi IS NULL;
GO
```

```

-- Fact_Anggaran → Dim_ProgramStudi
SELECT COUNT(*) AS Orphan_FactAnggaran_Prodi
FROM dbo.Fact_Anggaran fa
LEFT JOIN dbo.Dim_ProgramStudi dp ON fa.sk_prodi = dp.sk_prodi
WHERE dp.sk_prodi IS NULL;
GO

-- Fact_Akreditasi → Dim_Akreditasi
SELECT COUNT(*) AS Orphan_FactAkreditasi
FROM dbo.Fact_Akreditasi fa
LEFT JOIN dbo.Dim_Akreditasi da ON fa.sk_akreditasi = da.sk_akreditasi
WHERE da.sk_akreditasi IS NULL;
GO

-- Range Checks
-- Akreditasi Score should be between 0 and 4
SELECT *
FROM dbo.Dim_Akreditasi
WHERE nilai_akreditasi < 0 OR nilai_akreditasi > 4;
GO

-- Anggaran should never be negative
SELECT *
FROM dbo.Fact_Anggaran
WHERE total_anggaran < 0;
GO

-- Timeliness
SELECT
    MIN(tahun) AS tahun_min,
    MAX(tahun) AS tahun_max
FROM dbo.Dim_Waktu;
GO

-- SCD Validity Checks
-- Valid SCD1 Check: Ensure no duplicate business keys
SELECT nim
FROM dbo.Dim_Mahasiswa
GROUP BY nim
HAVING COUNT(*) > 1;
GO

```

6.2. Create Data Quality Dashboard

6.2.1. Data Profiling Metrics

- Jumlah baris per tabel (dim dan fact)

- Missing values count per kolom
 - Duplicate key count
 - Orphan fact records
 - Tipe data tidak valid (out-of-range)
- 6.2.2. SCD Consistency
- Jumlah mahasiswa IsCurrent=1
 - Jumlah mahasiswa dengan riwayat perubahan
 - Overlapping effective date ranges
- 6.2.3. Data Freshness
- Tanggal terakhir staging update
 - Tanggal terakhir dimension load
 - Tanggal terakhir fact load
- 6.2.4. Integrity & Validity Score
- Data Completeness Score (0-100%), formula: $1 - (\text{Total Missing Values} / \text{Total Expected Values})$
 - Data Consistency Score (0-100%), formula: $1 - (\text{Orphan Keys} + \text{Duplicates}) / \text{Total Rows}$
 - Data Freshness Score (0-100%), formula: $1 - (\text{Hari sejak last_load} / \text{threshold})$

a. Tabel audit untuk tracking data quality metrics

```
CREATE TABLE etl.DataQuality_Audit (
    audit_id INT IDENTITY(1,1) PRIMARY KEY,
    check_name VARCHAR(200) NOT NULL,
    table_name VARCHAR(200) NULL,
    issue_count INT NOT NULL DEFAULT 0,
    severity VARCHAR(20) NOT NULL, -- INFO / WARNING /
    CRITICAL
    status VARCHAR(10) NOT NULL, -- PASS / FAIL
    check_timestamp DATETIME DEFAULT GETDATE()
);
GO

CREATE TABLE dbo.DataQuality_Thresholds (
    check_name VARCHAR(200) PRIMARY KEY,
    threshold_value INT,
    severity VARCHAR(20), -- WARNING or CRITICAL
    description VARCHAR(300)
);
GO
```

b. Stored procedure untuk generate quality report

```

CREATE SCHEMA dq;
GO

CREATE PROCEDURE dq.Run_Data_Quality_Checks
AS
BEGIN
    SET NOCOUNT ON;

    -----
    -- 1. Missing NIM
    -----

    DECLARE @MissingNIM INT =
        (SELECT COUNT(*) FROM dbo.Dim_Mahasiswa WHERE nim IS
        NULL OR nim = "");

    INSERT INTO dbo.DataQuality_Audit (check_name, severity,
    result_value, status)
    SELECT
        'MissingNIM',
        t.severity,
        @MissingNIM,
        CASE WHEN @MissingNIM <= t.threshold_value THEN 'PASS'
    ELSE 'FAIL' END
    FROM dbo.DataQuality_Thresholds t
    WHERE t.check_name = 'MissingNIM';

    -----
    -- 2. Missing Tanggal Prestasi
    -----

    DECLARE @MissingTanggalPrestasi INT =
        (SELECT COUNT(*) FROM stg.Prestasi WHERE tanggal_prestasi
    IS NULL);

    INSERT INTO dbo.DataQuality_Audit
    SELECT
        'MissingTanggalPrestasi',
        t.severity,
        @MissingTanggalPrestasi,
        CASE WHEN @MissingTanggalPrestasi <= t.threshold_value
    THEN 'PASS' ELSE 'FAIL' END
    FROM dbo.DataQuality_Thresholds t
    WHERE t.check_name = 'MissingTanggalPrestasi';

    -----

```


-- 3. Duplicate Mahasiswa

```
-----  
DECLARE @DuplicateMahasiswa INT =  
(SELECT COUNT(*)  
FROM (  
    SELECT nim FROM dbo.Dim_Mahasiswa GROUP BY nim  
HAVING COUNT(*) > 1  
) x);  
  
INSERT INTO dbo.DataQuality_Audit  
SELECT  
    'DuplicateMahasiswa',  
    t.severity,  
    @DuplicateMahasiswa,  
    CASE WHEN @DuplicateMahasiswa <= t.threshold_value THEN  
'PASS' ELSE 'FAIL' END  
FROM dbo.DataQuality_Thresholds t  
WHERE t.check_name = 'DuplicateMahasiswa';
```

-- 4. Orphans: Fact_Prestasi → Dim_Mahasiswa

```
-----  
DECLARE @OrphanPrestasiMahasiswa INT =  
(SELECT COUNT(*)  
FROM dbo.Fact_Prestasi fp  
LEFT JOIN dbo.Dim_Mahasiswa dm ON fp.sk_mahasiswa =  
dm.sk_mahasiswa  
WHERE dm.sk_mahasiswa IS NULL);  
  
INSERT INTO dbo.DataQuality_Audit  
SELECT  
    'Orphan_FactPrestasi_Mahasiswa',  
    t.severity,  
    @OrphanPrestasiMahasiswa,  
    CASE WHEN @OrphanPrestasiMahasiswa <= t.threshold_value  
THEN 'PASS' ELSE 'FAIL' END  
FROM dbo.DataQuality_Thresholds t  
WHERE t.check_name = 'Orphan_FactPrestasi_Mahasiswa';
```

-- 5. Negative Budget Check

```
-----  
DECLARE @NegativeAnggaran INT =  
(SELECT COUNT(*) FROM dbo.Fact_Anggaran WHERE
```

```

total_anggaran < 0);

INSERT INTO dbo.DataQuality_Audit
SELECT
    'NegativeAnggaran',
    t.severity,
    @NegativeAnggaran,
    CASE WHEN @NegativeAnggaran <= t.threshold_value THEN
'PASS' ELSE 'FAIL' END
FROM dbo.DataQuality_Thresholds t
WHERE t.check_name = 'NegativeAnggaran';

END;
GO

```

- c. Alert jika quality threshold tidak terpenuhi

Menggunakan SQL Server Agent:

```

IF EXISTS (
    SELECT 1 FROM etl.DataQuality_Audit
    WHERE issue_count > 0
)
BEGIN
    RAISERROR ('DATA QUALITY ISSUE DETECTED!', 16, 1);
END

```

- d. Quality Metrics Summary

Kategori	Metrik	Target	Hasil
Completeness	Missing Values	0%	FAIL
Completeness	Duplicate Keys	0	PASS
Consistency	Orphan Fact Rows	0	PASS
Consistency	SCD Validity	1 row IsCurrent per NIM	PASS
Timeliness	Data Freshness	< 1 hari dari load terakhir	PASS
Validity	Range Checks (Anggaran,	Dalam batas	PASS

	Akreditasi)		
--	-------------	--	--

7. Step 7: Performance Testing

Tujuan: Menguji dan mengoptimalkan performa query

Aktivitas:

7.1. Create Test Queries

```
-- Simple Aggregation Tests
SELECT
    w.tahun,
    COUNT(fp.sk_prestasi) AS total_prestasi
FROM Fact_Prestasi fp
JOIN Dim_Waktu w
    ON fp.sk_waktu = w.sk_waktu
GROUP BY w.tahun
ORDER BY w.tahun;

SELECT
    dp.nama_prodi,
    SUM(fa.total_anggaran) AS total_anggaran
FROM Fact_Anggaran fa
JOIN Dim_ProgramStudi dp
    ON dp.sk_prodi = fa.sk_prodi
GROUP BY dp.nama_prodi
ORDER BY dp.nama_prodi;

-- Complex Join Tests
SELECT
    dm.nama_mahasiswa,
    dp.nama_prodi,
    w.tahun,
    COUNT(fp.sk_prestasi) AS jumlah_prestasi
FROM Fact_Prestasi fp
JOIN Dim_Mahasiswa dm ON fp.sk_mahasiswa = dm.sk_mahasiswa
JOIN Dim_ProgramStudi dp ON fp.sk_prodi = dp.sk_prodi
JOIN Dim_Waktu w ON fp.sk_waktu = w.sk_waktu
GROUP BY
    dm.nama_mahasiswa,
    dp.nama_prodi,
    w.tahun
ORDER BY
    dm.nama_mahasiswa,
    w.tahun;

-- Drill-down Analysis
```

```

SELECT
    dp.nama_prodi,
    w.tahun,
    w.semester,
    AVG(fak.rata_rata_ipk) AS ipk_rata_rata
FROM Fact_Akademik fak
JOIN Dim_ProgramStudi dp ON dp.sk_prodi = fak.sk_prodi
JOIN Dim_Waktu w ON w.sk_waktu = fak.sk_waktu
GROUP BY
    dp.nama_prodi,
    w.tahun,
    w.semester
ORDER BY
    dp.nama_prodi,
    w.tahun,
    w.semester;

-- Full Scan Report
SELECT *
FROM Fact_Anggaran
ORDER BY sk_waktu, sk_prodi;

-- Index Stress Test
SELECT
    dp.nama_prodi,
    SUM(fd.jumlah_dosen) AS total_dosen
FROM Fact_Dosen fd
JOIN Dim_ProgramStudi dp
    ON dp.sk_prodi = fd.sk_prodi
GROUP BY dp.nama_prodi
ORDER BY dp.nama_prodi;

```

7.2. Analyze Query Plans

a. Review execution plans

Periksa:

- apakah query menggunakan **Columnstore Index**
- apakah menggunakan **Index Seek** atau malah **Scan**
- estimasi vs actual row counts

b. Identify missing indexes

c. Check for table scans

Query Type	Target	Actual	Status
Simple	<1s	-	-

Aggregation			
Complex Join	<3s	-	-
Drill-down Analysis	<2s	-	-
Full Scan Report	<10s	-	-

d. Optimize join orders

7.3. Performance Benchmarks

Query Type	Target	Actual	Status
Simple Aggregation	<1s	-	-
Complex Join	<3s	-	-
Drill-down Analysis	<2s	-	-
Full Scan Report	<10s	-	-

7.4. Query Optimization Recommendations

7.4.1. Menambahkan Nonclustered Index

7.4.2. Mengganti Nested Loop→ Hash Join

7.4.3. Memastikan Partition Pruning

Cek apakah kondisi WHERE menggunakan sk_waktu:

```
WHERE w.tahun = 2023
```

Maka, SQL Server dapat melakukan prune partition.

7.4.4. Menambah Covering Index

7.4.5. Periksa Cardinality Estimation

```
ALTER DATABASE SCOPED CONFIGURATION CLEAR  
PROCEDURE_CACHE;
```